

Introducción

El objetivo de esta práctica fue desplegar y analizar la aplicación Mexi Banco v05.1 mediante Docker Compose, utilizando una arquitectura monolítica compuesta por una aplicación Java y una base de datos PostgreSQL. A través de la colección proporcionada para Postman se realizaron operaciones sobre cuentas, transferencias, movimientos y operaciones SPEI, además de diferentes pruebas de validación y manejo de errores.

La práctica también permitió observar el comportamiento de la persistencia de datos al detener y eliminar los contenedores, así como analizar de manera general la estructura interna del código fuente y la configuración utilizada para ejecutar los servicios.

Preparación y despliegue de la aplicación

Para comenzar, se utilizó la versión v05.1 del repositorio de Mexi Banco y posteriormente se inició la aplicación mediante Docker Compose.

El despliegue se realizó con:

```
docker compose up -d
```

Posteriormente se verificó el estado de los servicios mediante:

```
docker compose ps
```

Se comprobó que los servicios correspondientes a la aplicación y a la base de datos se encontraban activos y en estado healthy. Esto permitió confirmar que PostgreSQL había iniciado correctamente y que la aplicación podía ejecutarse sobre esta base de datos.

```
christian@pc-bda-cfr:~/GodsDocs2/UNIVERSIDAD/9noSemestre/SD/sd-2027-1/entregas/franco_christian/s05/mexi-ban
co$ docker compose up -d
[+] up 2/2
✓ Container mexi-banco-db-1 Healthy 0.5s
✓ Container mexi-banco-app-1 Started 7.9s
christian@pc-bda-cfr:~/GodsDocs2/UNIVERSIDAD/9noSemestre/SD/sd-2027-1/entregas/franco_christian/s05/mexi-ban
co$ docker compose ps
```

NAME	PORTS	IMAGE	COMMAND	SERVICE	CREATED	STATUS
mexi-banco-app-1	mexi-banco-app	"java -jar app.jar"	app	6 minutes ago	Up About a minute	
(healthy)	0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp					
mexi-banco-db-1	postgres:17-alpine	"docker-entrypoint.s..."	db	6 minutes ago	Up 6 minutes (hea	

Como comprobación adicional se consultó el endpoint de salud de la aplicación mediante Postman:

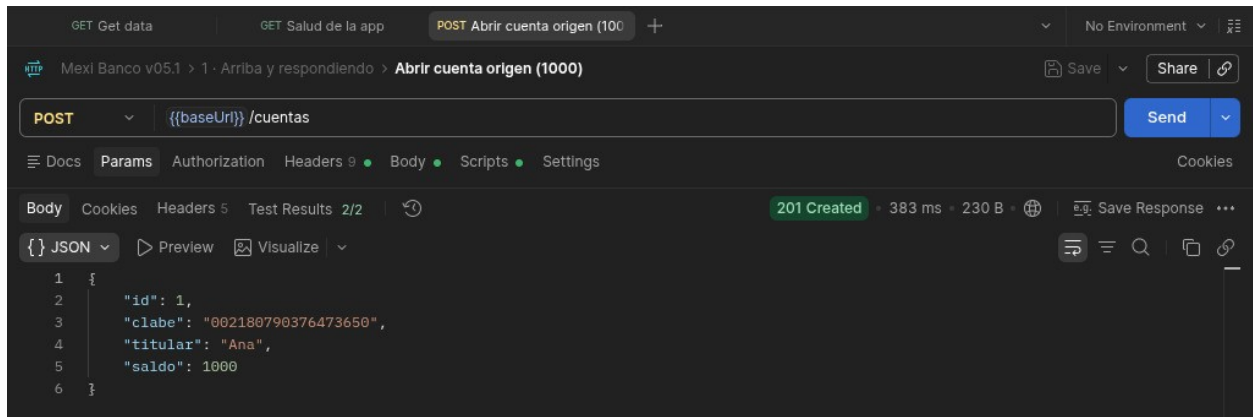
GET /actuador/health

La respuesta indicó que el estado de la aplicación era UP, lo que confirmó que el servicio estaba disponible para recibir solicitudes.

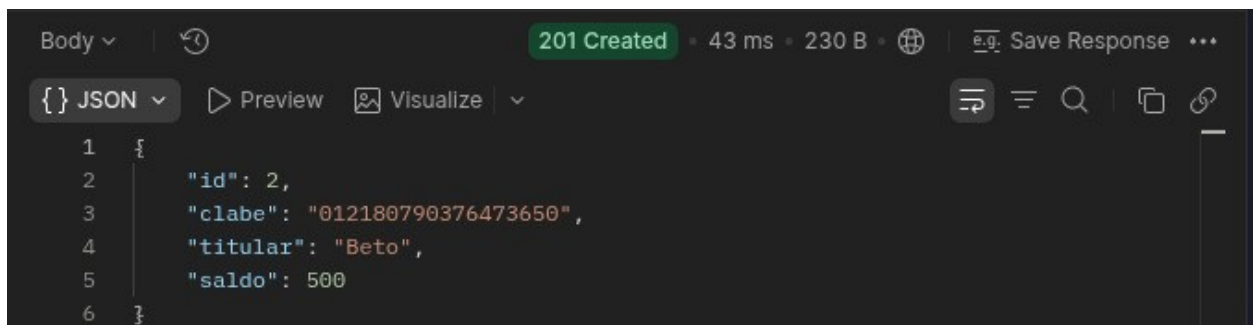
Operaciones sobre cuentas y transferencias

Una vez que el sistema se encontraba funcionando, se ejecutaron las operaciones definidas en la colección de Postman.

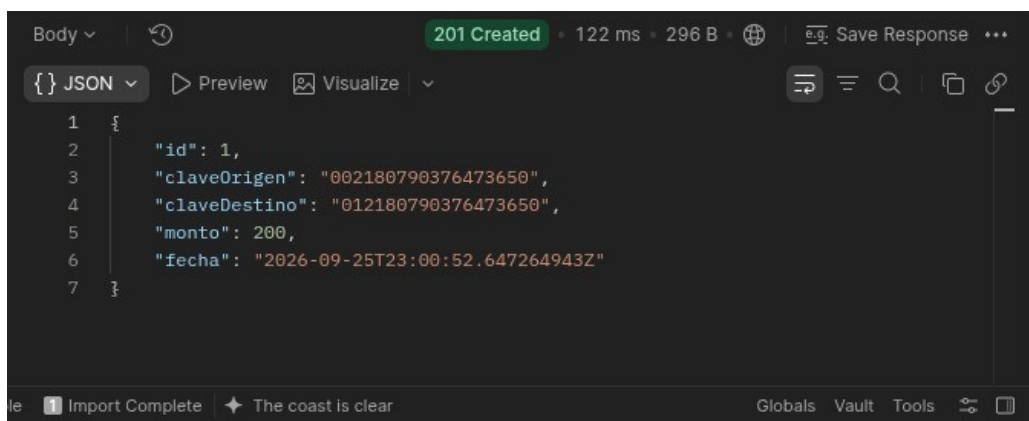
Primero se creó la cuenta de origen con un saldo inicial de 1000 unidades. Posteriormente se consultó la cuenta para verificar que el registro se hubiera creado correctamente y que el saldo inicial coincidiera con el esperado.

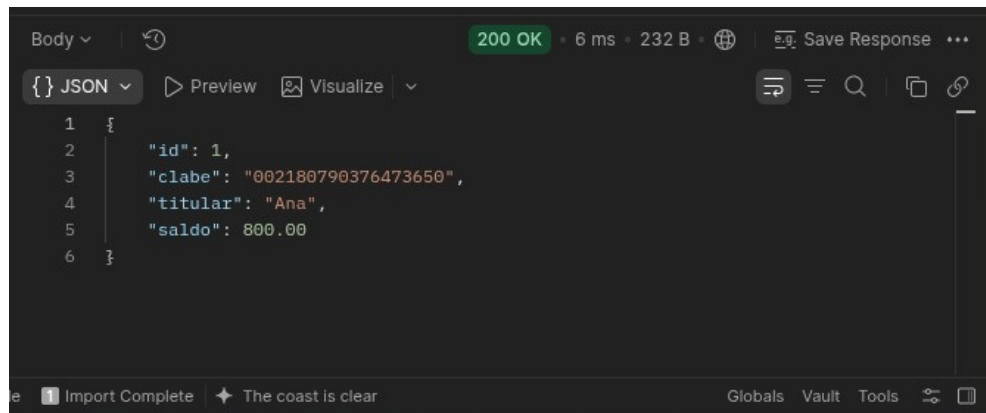


Después se creó una segunda cuenta, correspondiente al destino, con un saldo inicial de 500.



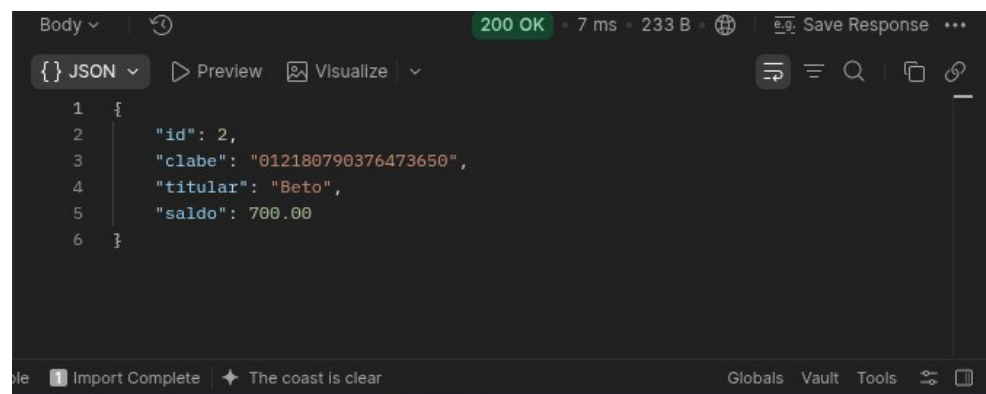
Posteriormente se realizó una transferencia interna de 200 desde la cuenta de origen hacia la cuenta destino. Como resultado, el saldo de la cuenta de origen disminuyó a 800 y el de la cuenta destino aumentó a 700.





```
Body ▾ | ↺ 200 OK • 6 ms • 232 B • 🌐 | 📄 Save Response •••  
{} JSON ▾ | ▶ Preview | 🖼 Visualize ▾ | 📄 📄 🔍 | 📄 🔗  
1 {  
2   "id": 1,  
3   "clabe": "002180790376473650",  
4   "titular": "Ana",  
5   "saldo": 800.00  
6 }
```

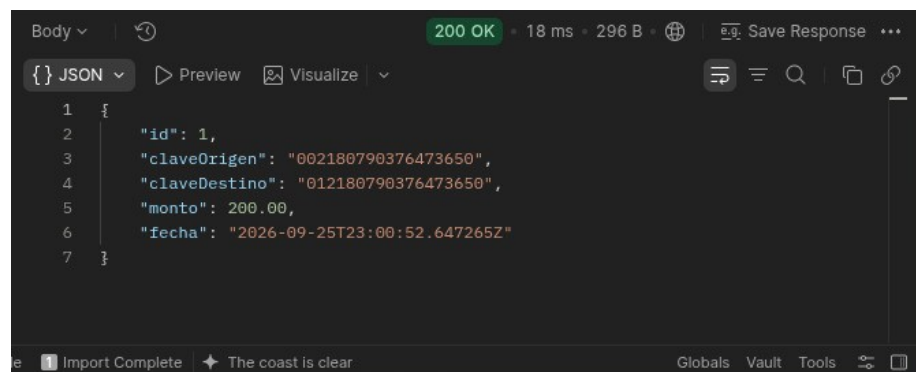
Import Complete | The coast is clear | Globals Vault Tools



```
Body ▾ | ↺ 200 OK • 7 ms • 233 B • 🌐 | 📄 Save Response •••  
{} JSON ▾ | ▶ Preview | 🖼 Visualize ▾ | 📄 📄 🔍 | 📄 🔗  
1 {  
2   "id": 2,  
3   "clabe": "012180790376473650",  
4   "titular": "Beto",  
5   "saldo": 700.00  
6 }
```

Import Complete | The coast is clear | Globals Vault Tools

Finalmente, se consultó la transferencia mediante su identificador para comprobar que la operación hubiese quedado registrada correctamente.



```
Body ▾ | ↺ 200 OK • 18 ms • 296 B • 🌐 | 📄 Save Response •••  
{} JSON ▾ | ▶ Preview | 🖼 Visualize ▾ | 📄 📄 🔍 | 📄 🔗  
1 {  
2   "id": 1,  
3   "claveOrigen": "002180790376473650",  
4   "claveDestino": "012180790376473650",  
5   "monto": 200.00,  
6   "fecha": "2026-09-25T23:00:52.647265Z"  
7 }
```

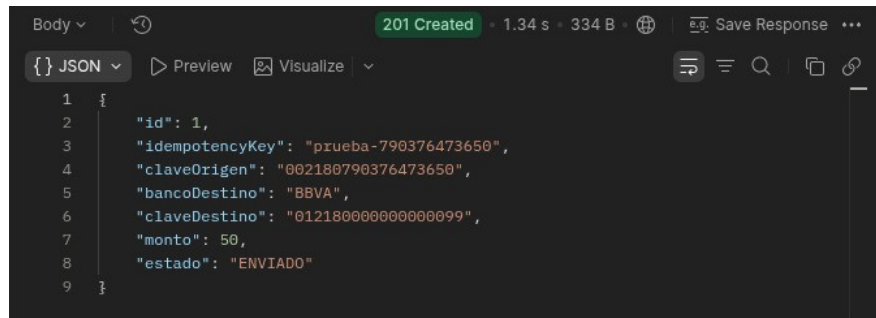
Import Complete | The coast is clear | Globals Vault Tools

Estas pruebas permitieron comprobar que una transferencia interna modifica los saldos de las cuentas involucradas y que la operación queda registrada para su posterior consulta.

Prueba de idempotencia en operaciones SPEI

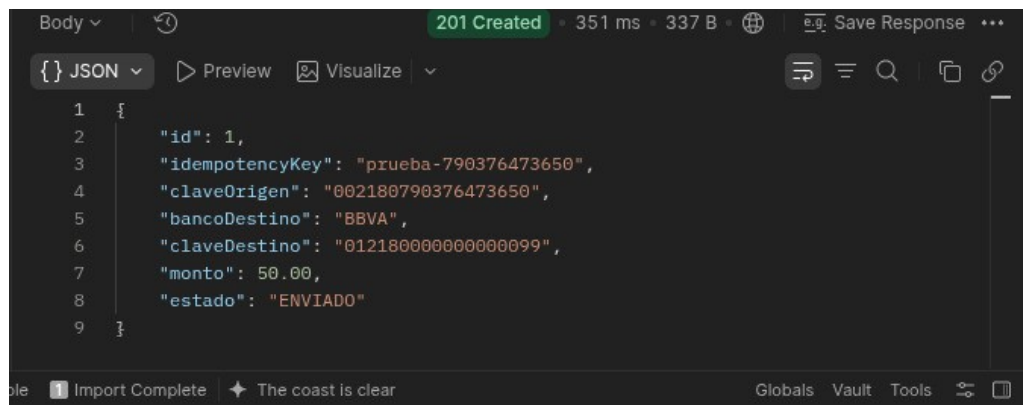
A continuación se realizó una operación SPEI por un monto de 50. Para esta petición se utilizó el encabezado **Idempotency-Key**, el cual permite identificar solicitudes repetidas y evitar que una misma operación sea procesada más de una vez.

El primer envío fue aceptado por la aplicación y quedó registrado con estado **ENVIADO**.



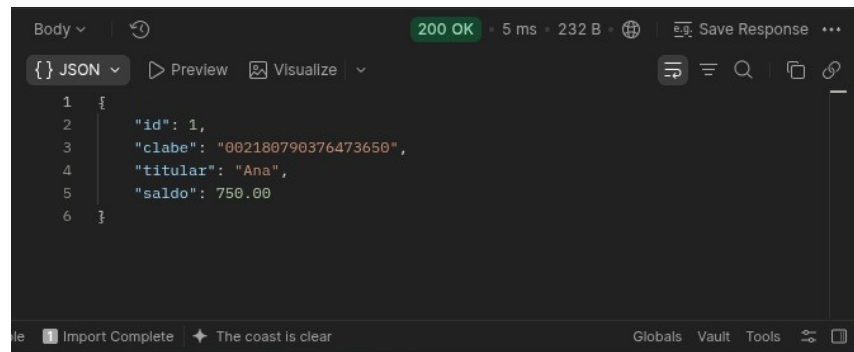
```
1 {
2   "id": 1,
3   "idempotencyKey": "prueba-790376473650",
4   "claveOrigen": "002180790376473650",
5   "bancoDestino": "BBVA",
6   "claveDestino": "012180000000000099",
7   "monto": 50,
8   "estado": "ENVIADO"
9 }
```

Posteriormente se repitió exactamente la misma solicitud utilizando la misma clave de idempotencia. La aplicación devolvió nuevamente una respuesta exitosa y mantuvo el mismo identificador de la operación, mostrando que el segundo envío no generó una nueva transacción.



```
1 {
2   "id": 1,
3   "idempotencyKey": "prueba-790376473650",
4   "claveOrigen": "002180790376473650",
5   "bancoDestino": "BBVA",
6   "claveDestino": "012180000000000099",
7   "monto": 50.00,
8   "estado": "ENVIADO"
9 }
```

Después del reintento, se consultó nuevamente la cuenta de origen. El saldo resultó ser de 750, en lugar de disminuir nuevamente a 700.

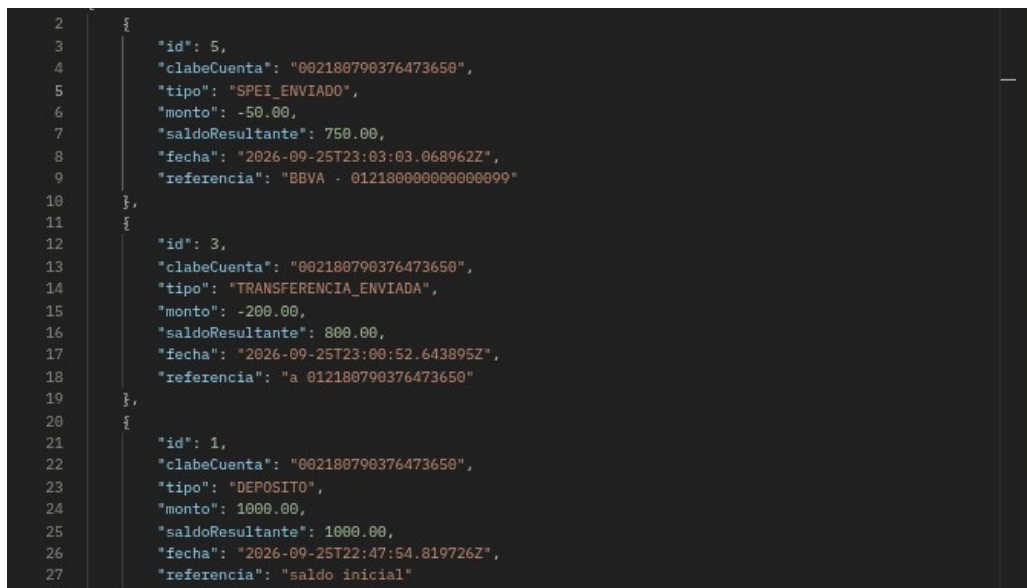


La prueba demuestra el uso de idempotencia para hacer que una solicitud repetida sea segura. En este caso, aunque la petición SPEI se envió dos veces, el movimiento económico solamente tuvo efecto una vez.

Consulta de movimientos y notificaciones

Con las operaciones anteriores realizadas, se consultó el estado de cuenta de la cuenta de origen mediante el endpoint de movimientos.

La consulta permitió observar los movimientos generados por las operaciones realizadas. En particular, se verificó la presencia del movimiento correspondiente al SPEI enviado y que el estado de cuenta fuera consistente con el saldo final de 750.



También se consultaron las notificaciones asociadas a la cuenta. La respuesta permitió comprobar que el sistema genera avisos relacionados con las operaciones realizadas, incluyendo la transferencia y el SPEI.

```

1 [
2   {
3     "id": 3,
4     "clabeCuenta": "002180790376473650",
5     "mensaje": "SPEI_ENVIADO por 50, saldo resultante 750.00",
6     "fecha": "2026-09-25T23:03:03.477422Z"
7   },
8   {
9     "id": 1,
10    "clabeCuenta": "002180790376473650",
11    "mensaje": "TRANSFERENCIA ENVIADA por 200, saldo resultante 800.00",
12    "fecha": "2026-09-25T23:00:52.650809Z"
13  }
14 ]

```

Estas consultas muestran que las operaciones bancarias no únicamente modifican el saldo, sino que también generan información histórica y notificaciones asociadas al usuario.

Manejo de errores y validaciones

Posteriormente se realizaron diferentes solicitudes diseñadas para provocar errores controlados y verificar que el sistema validara correctamente las operaciones.

Primero se consultó una CLABE inexistente, obteniendo una respuesta **404 Not Found**, lo que indica que el recurso solicitado no existe.

```

1 {
2   "type": "about:blank",
3   "title": "Not Found",
4   "status": 404,
5   "detail": "No existe la CLABE 99999999999999999999",
6   "instance": "/cuentas/99999999999999999999"
7 }

```

Después se intentó registrar nuevamente una cuenta utilizando una CLABE que ya existía. La aplicación respondió con **409 Conflict**, evitando la creación de un registro duplicado.

```

1 {
2   "type": "about:blank",
3   "title": "Conflict",
4   "status": 409,
5   "detail": "Ya existe una cuenta con la CLABE 002180790376473650",
6   "instance": "/cuentas"
7 }

```

También se realizó una solicitud para crear una cuenta sin proporcionar el campo obligatorio de CLABE. En este caso, el sistema respondió con **400 Bad Request**.

A screenshot of a REST client interface showing a 400 Bad Request response. The status bar at the top indicates "400 Bad Request" with a response time of 134 ms and a size of 233 B. The response body is displayed in JSON format:

```

1 {
2   "timestamp": "2026-09-25T23:06:37.274+00:00",
3   "status": 400,
4   "error": "Bad Request",
5   "path": "/cuentas"
6 }

```

Para comprobar las validaciones relacionadas con las transferencias, se intentó transferir una cantidad superior al saldo disponible. La aplicación respondió con **422 Unprocessable Entity**, impidiendo que la operación pudiera ejecutarse.

A screenshot of a REST client interface showing a 422 Unprocessable Entity (WebDAV) (RFC 4918) response. The status bar at the top indicates "422 Unprocessable Entity (WebDAV) (RFC 4918)" with a response time of 16 ms and a size of 328 B. The response body is displayed in JSON format:

```

1 {
2   "type": "about:blank",
3   "title": "Unprocessable Entity",
4   "status": 422,
5   "detail": "Saldo insuficiente en la CLABE 002180790376473650",
6   "instance": "/transferencias"
7 }

```

De manera similar, se intentó realizar una transferencia hacia la misma cuenta de origen. Esta operación también fue rechazada con una respuesta **422**.

A screenshot of a REST client interface showing a 422 Unprocessable Entity (WebDAV) (RFC 4918) response. The status bar at the top indicates "422 Unprocessable Entity (WebDAV) (RFC 4918)" with a response time of 4 ms and a size of 321 B. The response body is displayed in JSON format:

```

1 {
2   "type": "about:blank",
3   "title": "Unprocessable Entity",
4   "status": 422,
5   "detail": "La cuenta origen y la destino son la misma",
6   "instance": "/transferencias"
7 }

```

En el caso de una operación SPEI sin proporcionar el encabezado Idempotency-Key, la aplicación devolvió **400 Bad Request**, demostrando que este dato es obligatorio para dicha operación.

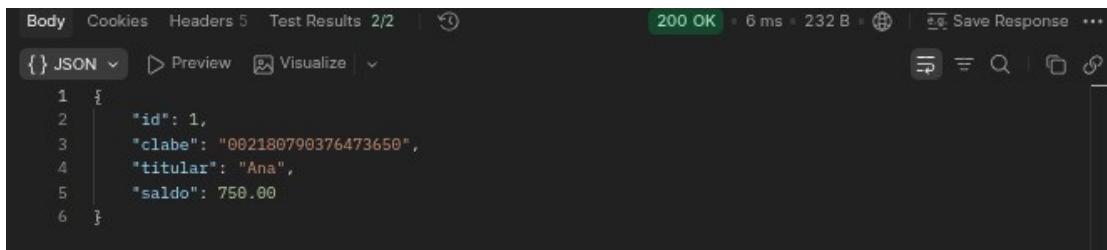
A screenshot of a REST client interface showing a 400 Bad Request response. The status bar at the top indicates "400 Bad Request" with a response time of 4 ms and a size of 230 B. The response body is displayed in JSON format:

```

1 {
2   "timestamp": "2026-09-25T23:07:38.569+00:00",
3   "status": 400,
4   "error": "Bad Request",
5   "path": "/spei"
6 }

```

Finalmente, se volvió a consultar la cuenta de origen para comprobar que las operaciones rechazadas no hubieran modificado el saldo. El saldo permaneció en 750.



Estas pruebas muestran que la aplicación utiliza códigos HTTP y validaciones específicas para controlar diferentes situaciones inválidas sin alterar el estado correcto de las cuentas.

Persistencia de datos con Docker Compose

Una de las pruebas principales de la práctica consistió en analizar el comportamiento de los datos al detener y volver a iniciar los servicios.

Primero se ejecutó:

```
docker compose down
```

Este comando eliminó los contenedores, pero mantuvo el volumen asociado a PostgreSQL. Después se inició nuevamente la aplicación mediante:

```
docker compose up -d
```

Una vez que los servicios volvieron a estar disponibles, se consultó nuevamente la cuenta de origen. Los datos previamente registrados continuaron disponibles, incluyendo el saldo de 750.

```
christian@pc-bda-cfr:~/GodsDocs2/UNIVERSIDAD/9noSemestre/SD/sd-2027-1/entregas/franco_christian/s05/mexi-ban
co$ docker compose ps
NAME                IMAGE             COMMAND                  SERVICE   CREATED         STATUS
mexi-banco-app-1    mexi-banco-app    "java -jar app.jar"      app       42 minutes ago  Up 36 minutes (h
ealthy)   0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
mexi-banco-db-1     postgres:17-alpine "docker-entrypoint.s..." db        42 minutes ago  Up 41 minutes (h
ealthy)   5432/tcp
christian@pc-bda-cfr:~/GodsDocs2/UNIVERSIDAD/9noSemestre/SD/sd-2027-1/entregas/franco_christian/s05/mexi-ban
co$ docker compose down
[+] down 3/3
✓ Container mexi-banco-app-1 Removed 19.0s
✓ Container mexi-banco-db-1 Removed 13.8s
✓ Network mexibanco Removed 3.0s
christian@pc-bda-cfr:~/GodsDocs2/UNIVERSIDAD/9noSemestre/SD/sd-2027-1/entregas/franco_christian/s05/mexi-ban
co$ docker compose up -d
[+] up 3/3
✓ Network mexibanco Created 1.5s
✓ Container mexi-banco-db-1 Healthy 30.9s
✓ Container mexi-banco-app-1 Started 28.2s
christian@pc-bda-cfr:~/GodsDocs2/UNIVERSIDAD/9noSemestre/SD/sd-2027-1/entregas/franco_christian/s05/mexi-ban
co$
```

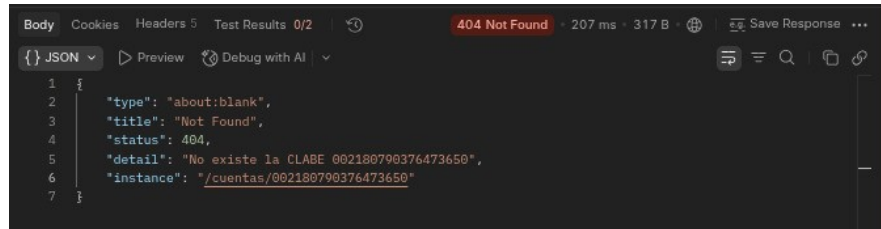
Posteriormente se ejecutó:

```
docker compose down -v
```

En este caso, además de eliminar los contenedores, se eliminaron los volúmenes asociados al proyecto. La aplicación se volvió a iniciar con:

```
docker compose up -d
```

Al consultar nuevamente la cuenta de origen, ésta ya no se encontró y la aplicación respondió con **404 Not Found**.



A partir de esta prueba se puede observar la diferencia entre ambos comandos. `docker compose down` elimina los contenedores pero conserva el almacenamiento persistente, mientras que `docker compose down -v` elimina también los volúmenes y, por lo tanto, los datos almacenados en PostgreSQL.

Análisis de la estructura del monolito

Para finalizar la práctica se inspeccionó la estructura del código fuente ubicado en:

```
src/main/java/mx/mexibanco
```

La aplicación se encuentra organizada en distintos módulos funcionales, entre ellos:

```
cuenta
movimiento
transferencia
notificacion
spei
```

En el caso del módulo `cuenta`, se identificaron archivos como:

```
Cuenta.java
CuentaRepository.java
CuentaService.java
CuentaController.java
```

Esta organización permite separar las responsabilidades principales de la aplicación. El controlador gestiona las solicitudes HTTP, el servicio concentra la lógica de negocio, el repositorio se encarga del acceso a los datos y la clase de cuenta representa la información correspondiente a la entidad.

```

christian@pc-bda-cfr:~/GodsDocs2/UNIVERSIDAD/9noSemestre/SD/sd-2027-1/entregas/franco_christian/s05/mexi-ban
co$ find src/main/java/mx/mexibanco -type f | sort
src/main/java/mx/mexibanco/compartido/ConflictoException.java
src/main/java/mx/mexibanco/compartido/ManejadorDeErrores.java
src/main/java/mx/mexibanco/compartido/NoEncontradoException.java
src/main/java/mx/mexibanco/compartido/ReglaDeNegocioException.java
src/main/java/mx/mexibanco/cuenta/CuentaController.java
src/main/java/mx/mexibanco/cuenta/Cuenta.java
src/main/java/mx/mexibanco/cuenta/CuentaRepository.java
src/main/java/mx/mexibanco/cuenta/CuentaService.java
src/main/java/mx/mexibanco/MexiBancoApplication.java
src/main/java/mx/mexibanco/movimiento/MovimientoController.java
src/main/java/mx/mexibanco/movimiento/Movimiento.java
src/main/java/mx/mexibanco/movimiento/MovimientoRepository.java
src/main/java/mx/mexibanco/movimiento/MovimientoService.java
src/main/java/mx/mexibanco/movimiento/TipoMovimiento.java
src/main/java/mx/mexibanco/notificacion/NotificacionController.java
src/main/java/mx/mexibanco/notificacion/Notificacion.java
src/main/java/mx/mexibanco/notificacion/NotificacionRepository.java
src/main/java/mx/mexibanco/notificacion/NotificacionService.java
src/main/java/mx/mexibanco/spei/EstadoSpei.java
src/main/java/mx/mexibanco/spei/SpeiController.java
src/main/java/mx/mexibanco/spei/SpeiService.java
src/main/java/mx/mexibanco/spei/SpeiSolicitud.java
src/main/java/mx/mexibanco/spei/SpeiSolicitudRepository.java
src/main/java/mx/mexibanco/spei/SpeiTransacciones.java
src/main/java/mx/mexibanco/transferecia/TransfereciaController.java
src/main/java/mx/mexibanco/transferecia/Transferecia.java
src/main/java/mx/mexibanco/transferecia/TransfereciaRepository.java
src/main/java/mx/mexibanco/transferecia/TransfereciaService.java
christian@pc-bda-cfr:~/GodsDocs2/UNIVERSIDAD/9noSemestre/SD/sd-2027-1/entregas/franco_christian/s05/mexi-ban
co$

```

También se revisó la existencia de los módulos relacionados con movimientos, transferencias, notificaciones y SPEI, observando que cada uno implementa una parte específica de la lógica del banco.

Esta estructura permite identificar el funcionamiento general del sistema como un monolito, ya que los diferentes componentes funcionales forman parte de una misma aplicación y se despliegan conjuntamente.

Conclusiones

La práctica permitió analizar el funcionamiento de una aplicación bancaria monolítica desplegada con Docker Compose, desde su inicialización hasta la ejecución de operaciones y validaciones mediante una API REST.

Las pruebas de cuentas y transferencias permitieron comprobar la modificación de saldos y el registro de operaciones. La prueba de idempotencia mostró cómo una misma solicitud SPEI puede repetirse sin generar un segundo efecto económico cuando se utiliza la misma clave de idempotencia.

Las pruebas de error permitieron observar que el sistema valida diferentes condiciones y utiliza códigos HTTP específicos para comunicar resultados como recursos inexistentes, conflictos, solicitudes inválidas y operaciones no permitidas.

Finalmente, el experimento con los volúmenes de Docker permitió comprobar la importancia de la persistencia de datos. Al utilizar `docker compose down`, los datos permanecieron disponibles debido a que el volumen de PostgreSQL no fue eliminado. En cambio, con `docker compose down -v`, el volumen también fue eliminado y la base de datos inició nuevamente sin los registros anteriores.

En conjunto, la práctica permitió relacionar conceptos de persistencia, APIs, validación, idempotencia, contenedores y arquitectura monolítica dentro de un mismo sistema distribuido.